

CS 499 Milestone Four Narrative for

Enhancement Three: Databases by John Grudzinski

Artifact Description

The artifact I selected for the databases category is my CS 340 Grazioso Salvare MongoDB/Dash dashboard. I originally created this artifact in my CS 340 Advanced Programming Concepts class. It includes a Python CRUD module that connects to a MongoDB database containing Austin Animal Center animal shelter records and a Dash dashboard that allows the user to filter records, view animal data in a table, display a breed distribution chart, and view map markers based on animal locations. The purpose of the original project was to help Grazioso Salvare identify dogs that could be strong candidates for different rescue/training categories such as water rescue, mountain or wilderness rescue, and disaster or individual tracking.

Justification for Inclusion

The reason I selected this artifact is because it directly demonstrates database access, data manipulation, filtering, data visualization, and client centered decision support. It also fits my career focus which is secure full stack software development because the project connects a user facing dashboard to a backend database through a reusable Python module. The original artifact already showed that I could connect to MongoDB, perform CRUD operations, and present data visually through Dash, pandas, Plotly, and Dash Leaflet. It is worth noting that the original version also had limitations which is why I thought it made a good candidate for enhancement. Most importantly, the MongoDB credentials were hard coded, the dashboard loaded the full

dataset before filtering in pandas, and the CRUD module could use stronger validation and more database-focused functionality.

Enhancement Summary

I enhanced the artifact by improving the database layer and the dashboard controller logic. In the enhanced CRUD module, I removed hard-coded default credentials and added support for environment variables such as AAC_USER, AAC_PASSWORD, AAC_HOST, AAC_PORT, AAC_DB, and AAC_COLLECTION. This makes the code safer to publish in an ePortfolio because real credentials no longer need to be stored directly in the source code. I also added stronger validation for create, read, update, delete, and aggregate operations. For example, update operations now validate that the update document uses MongoDB update operators such as \$set, \$inc, or \$unset, and delete operations still reject empty queries to reduce the risk of accidental mass deletion.

I also improved the dashboard from a database design perspective. The original dashboard used `db.read({})` to load all records into a pandas DataFrame and then filtered the data in Python. In the enhanced version, dashboard filter selections are translated into MongoDB queries. This means MongoDB performs more of the filtering before records reach the dashboard. I added projection support so the dashboard only requests the fields it needs for the table, chart, and map. I also added aggregation support through an `aggregate()` method and an `aggregate_breed_counts()` method so breed counts for the pie chart can be calculated in MongoDB rather than only through pandas `value_counts()`. Lastly, I added an index helper that creates or reuses indexes on fields commonly used by the dashboard filters, such as `animal_type`, `breed`, `sex_upon_outcome`, and `age_upon_outcome_in_weeks`.

Skills Demonstrated

This enhancement showcases several of my database related skills. For starters, it demonstrates database security awareness by removing credentials from the codebase and shifting configuration to environment variables. Second, it demonstrates improved database access patterns by using query based filtering, projection, and aggregation. Third, it demonstrates defensive programming because the enhanced CRUD module checks input types and rejects risky operations before they are sent to MongoDB. Fourth, it demonstrates the ability to connect database decisions to a working product. The enhanced dashboard still supports the original user goals, but the data flow is more scalable, secure, and maintainable than the original version.

Course Outcomes

Completing this enhancement makes progress toward several of the CS 499 course outcomes. It strongly supports Course Outcome 1 because the dashboard helps a client use data to support organizational decision making. Grazioso Salvare can use the dashboard to identify potential rescue dogs by rescue category, breed, sex, age range, and location. It also supports Course Outcome 4 because I used well-founded tools and techniques in Python, MongoDB, Dash, pandas, Plotly, and Dash Leaflet to implement a solution that delivers value to a real client scenario. Course Outcome 5 is also addressed because the enhancement reduces security risks by removing hard coded credentials, validating database operations, rejecting unsafe update/delete requests, and encouraging safer configuration practices. Course Outcome 2 is supported through this narrative and through clearer code comments and documentation that explain the technical decisions in a way that is understandable to a reviewer or instructor.

Reflection on the Enhancement Process

I learned more than a few things working on this enhancement but the most important thing I learned is that database design is not only about storing and retrieving data. It is also about deciding where work should happen, how much data should move between layers, and how to protect data access. In the original dashboard, filtering in pandas was straightforward and worked for my initial class project, but it was not the strongest design if the dataset grew or if the dashboard became a production style tool. Moving more filtering and aggregation into MongoDB helped me think more carefully about database responsibilities and the trade-offs between application-side and database-side processing.

The main challenge was preserving the original dashboard behavior while improving the backend design. I wanted the table, chart, and map to keep serving the same user needs, but I also wanted to improve the data flow behind them. Another challenge was credential handling because hard coded credentials are convenient for a class environment, but they are not appropriate for code that may be shared publicly. Using environment variables required a more professional setup, but it better reflects secure development practices. Overall, this enhancement helped me strengthen my database skills and gave me a more complete artifact for my ePortfolio because it now shows CRUD functionality, query design, aggregation, projection, indexing, validation, and security minded configuration.